

MODEL-DRIVEN DEVELOPMENT

UTILIZANDO LA HERRAMIENTA BOUML

EQUIPO B

¿QUÉ ES MDE?



MODELOS COMO ARTEFACTOS PRINCIPALES

MDE (Model-Driven Engineering) es una metodología de desarrollo de software que eleva los modelos al rol central del proceso, reemplazando el código como artefacto principal.



PRINCIPIOS FUNDAMENTALES DE MDE

- Abstracción: modelos independientes de la plataforma.
- Transformación: conversión automática entre modelos.
- Automatización: generación de código desde modelos.
- Interoperabilidad: integración entre herramientas y plataformas.
- Reutilización: componentes y patrones reutilizables.



¿QUÉ ES MDA?

CIM

- Modelo Independiente de Computación
- Define requisitos y contexto del negocio
- Sin detalles técnicos ni de implementación
 - Representa el "qué" del sistema
- Base para la comunicación con stakeholders

PIM

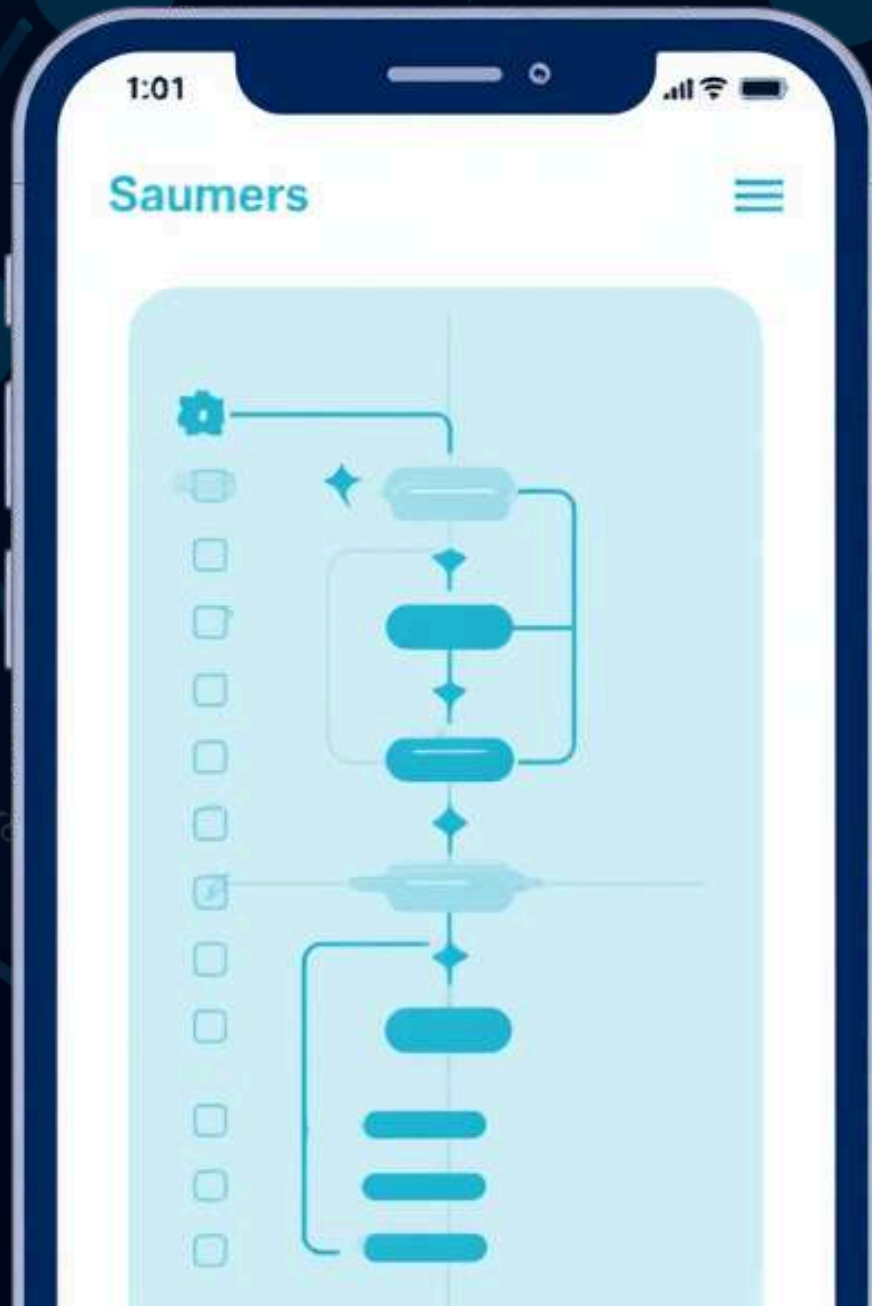
- Modelo Independiente de Plataforma
- Describe la lógica del sistema sin plataforma específica
- Independiente de tecnología de implementación
- Modelos UML reutilizables y portables
 - Transformado automáticamente desde el CIM

PSM

- Modelo Específico de Plataforma
- Adapta el PIM a una plataforma concreta
- Incluye detalles de Java, .NET, SQL, etc.
- Base para la generación de código fuente
- Último nivel antes de la implementación

¿QUÉ ES MDD Y CÓMO FUNCIONA?

- Metodología que usa modelos como artefactos principales del desarrollo de software
- Genera código fuente automáticamente a partir de modelos abstractos
- Aumenta la productividad al reducir la escritura manual de código
- Disminuye errores humanos mediante transformaciones automatizadas
- Flujo: Modelo abstracto → Transformación → Código ejecutable



QUÉ ES UML?

LENGUAJE DE MODELADO UNIFICADO

Estándar ISO para visualizar, especificar, construir y documentar sistemas de software. Define una notación gráfica común adoptada globalmente en ingeniería de software.

TIPOS DE DIAGRAMAS UML

- Estructurales: clases, componentes, despliegue.
- Comportamiento: casos de uso, actividad, estados.
- Interacción: secuencia, comunicación, tiempos.
- Base de MDD: modelos ejecutables y transformables.



MODELADO DE REQUISITOS

MODELO DE DATOS

- Define entidades, atributos y relaciones del sistema
- Representado mediante diagramas de clases UML
- Establece la estructura estática de la información
- Base para la generación de esquemas de base de datos
- Ejemplo: entidades Usuario, Producto, Pedido

MODELO FUNCIONAL

- Describe qué hace el sistema, no cómo lo hace
- Representado con diagramas de casos de uso UML
- Captura interacciones entre actores y el sistema
- Define los requisitos funcionales del software
- Ejemplo: Registrar usuario, Procesar pago

MODELO DE COMPORTAMIENTO

- Describe cómo el sistema responde a eventos
- Representado con diagramas de secuencia y estados
- Modela el flujo dinámico entre componentes
- Permite validar la lógica de negocio del sistema
- Ejemplo: flujo de autenticación y transiciones de estado

¿QUÉ ES BOUML?



HERRAMIENTA UML MULTIPLATAFORMA

Software de modelado UML libre y de código abierto. Compatible con Windows, Linux y macOS. Soporta UML 2.x con diagramas de clases, secuencia, casos de uso, actividad y estado.



GENERACIÓN DE CÓDIGO MULTILENGUAJE

Genera código en C++, Java, Python, PHP, C# e IDL. Permite ingeniería directa e inversa. Sincronización automática entre modelo y código fuente.



ALTO RENDIMIENTO Y EFICIENCIA

Diseñado para manejar proyectos de gran escala. Interfaz ligera con tiempos de respuesta rápidos. Plugins extensibles para personalizar funcionalidades.

VENTAJAS DE BOUML

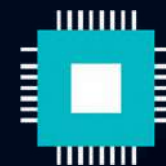


VELOCIDAD EXCEPCIONAL



Genera código fuente de forma casi instantánea. Procesa modelos UML complejos con tiempos de respuesta mínimos, superando ampliamente a herramientas equivalentes en rendimiento de generación.

BAJO CONSUMO DE RECURSOS



Diseño ligero y eficiente: consume muy poca memoria RAM y CPU. Funciona fluidamente en equipos con recursos limitados, sin requerir instalaciones pesadas ni dependencias complejas.

SOPORTE MULTILENGUAJE



Compatible con C++, Java, PHP, Python, MySQL y más. Permite generar e importar código en múltiples lenguajes desde un mismo modelo UML, facilitando proyectos multiplataforma.

BOUML VS. OTRAS HERRAMIENTAS

BOUML

- Interfaz menos intuitiva
- Curva de aprendizaje alta
- Documentación limitada
 - Soporte comunitario reducido
- Actualizaciones poco frecuentes

STARUML

- Gratuito y open source
 - Interfaz moderna y amigable
 - Multiplataforma (Win/Mac/Linux)
- Extensiones y plugins activos
- Comunidad amplia y activa

VISUAL PARADIGM

- Suite más completa y robusta
- Generación de código avanzada
- Colaboración en tiempo real
 - Soporte empresarial profesional
- Mayor integración con IDEs

FORWARD ENGINEERING

¿QUÉ ES LA INGENIERÍA DIRECTA?



- Transforma modelos UML en código ejecutable automáticamente
- Reduce errores manuales y acelera el desarrollo
- BOUML genera código en C++, Java, PHP, Python y más
- El modelo es la fuente principal de verdad del sistema

PROCESO DE TRANSFORMACIÓN



- PIM → PSM → Código: pipeline automatizado de generación
- Plantillas configurables adaptan el código a cada plataforma
- Sincronización entre diagrama y código en tiempo real
- Consistencia garantizada entre diseño e implementación



INGENIERÍA INVERSA Y ROUNDRIP ENGINEERING



INGENIERÍA INVERSA

- Genera modelos UML desde código fuente existente
- BOUML analiza código C++, Java, Python y más
- Recupera la arquitectura sin documentación previa
- Facilita la comprensión de sistemas legacy



ROUNDRIP ENGINEERING

- Sincronización bidireccional entre modelo y código
- Cambios en el código se reflejan en el modelo UML
- Cambios en el modelo regeneran el código actualizado
- Mantiene consistencia continua en el ciclo de vida



BENEFICIOS Y LIMITACIONES DE MDD

BENEFICIOS CLAVE

- Mayor productividad en el desarrollo
- Modelos reutilizables entre proyectos
- Mejor documentación del sistema
- Reducción de errores por abstracción
- Generación automática de código

LIMITACIONES

- Curva de aprendizaje pronunciada
 - Dependencia de herramientas específicas
- Modelos complejos difíciles de mantener
- Código generado puede ser ineficiente
 - Requiere expertos en metamodelado

CONSIDERACIONES FINALES

- MDD es ideal para proyectos grandes y complejos
 - Evaluar la madurez del equipo antes de adoptar
- Invertir en capacitación es fundamental
- Seleccionar herramientas adecuadas al contexto
- Balance entre abstracción y control del código

¡GRACIAS!

MDD CON BOUML OPTIMIZA EL DESARROLLO DE SOFTWARE
MEDIANTE MODELADO EFICIENTE



Conclusiones: MDD acelera el ciclo de vida del software



BOUML: herramienta potente, gratuita y multiplataforma